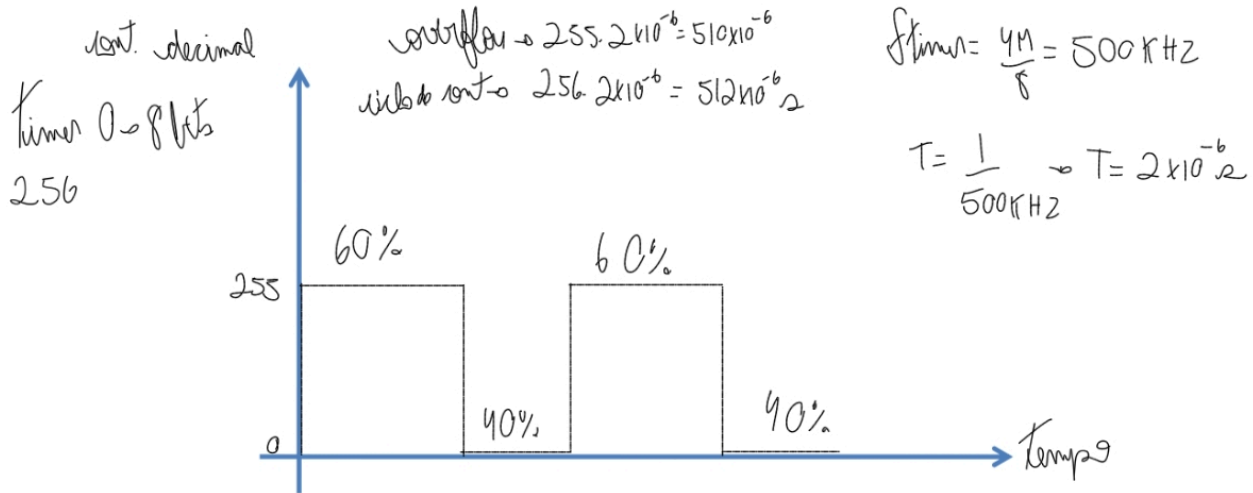
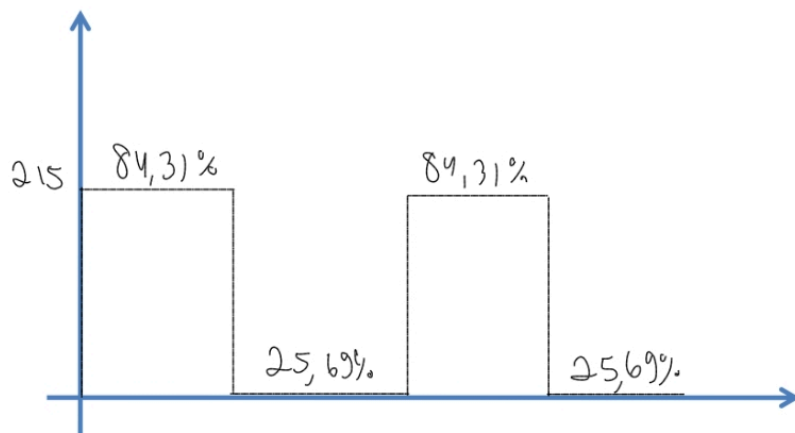


1) Desenhe o gráfico de contagem do TIMER 0 do ATmega328p, representando no eixo vertical o valor de contagem em decimal, e no eixo horizontal o tempo. Considere que esse temporizador esteja configurado para o modo normal de contagem. Considere também uma frequência de CPU de 4MHz e um prescaler com fator f/8.



2) Desenhe o gráfico de saída no pino OC0A referente à contagem acima, considerando OCR0A = 215. Qual o **Duty Cycle** do sinal resultante?



$$\text{Duty Cycle} = \frac{215}{255} \times 100 = 84,31\%$$

$$1 \cdot 215 \cdot 2 \times 10^{-6} = 430 \times 10^{-6}\text{s}$$

3. Dados:

F_CPU = 4 MHz

Timer usado = TIMER1

Modo = Fast PWM com ICR1 como TOP

Prescaler = 1

Saída PWM = OC1A / PB1

INT0 = PD2

INT1 = PD3

PWM ~48 kHz com INT1 e INT0

$$f_{PWM} = \frac{f_{CPU}}{N \cdot (1 + TOP)}$$

$$TOP = \frac{4.000.000}{1 \cdot 48.000} - 1 \approx 82,33$$

$$f_{PWM} = \frac{4.000.000}{1 \cdot (82 + 1)} = \frac{4.000.000}{83} \approx 48,19, \text{ kHz}$$

```
#define F_CPU 4000000UL

#include <avr/io.h>
#include <avr/interrupt.h>

#define TOP_PWM 82

volatile uint8_t duty = 0;

void atualizar_pwm(void)
{
    if (duty == 0)
    {
        TCCR1A &= ~(1 << COM1A1);
        PORTB &= ~(1 << PB1);
        OCR1A = 0;
    }
    else
    {
        TCCR1A |= (1 << COM1A1);
        OCR1A = ((uint32_t)TOP_PWM * duty) / 100;
    }
}

ISR(INT1_vect)
{
    if (duty <= 90)
        duty += 10;
    else
        duty = 100;

    atualizar_pwm();
}

ISR(INT0_vect)
```

```

{
    duty = 0;
    atualizar_pwm();
}

int main(void)
{
    DDRB |= (1 << PB1);
    DDRD &= ~( (1 << PD2) | (1 << PD3) );

    ICR1 = TOP_PWM;
    OCR1A = 0;

    TCCR1A = (1 << WGM11);
    TCCR1B = (1 << WGM13) | (1 << WGM12) | (1 << CS10);

    EICRA = (1 << ISC11) | (1 << ISC10) |
            (1 << ISC01) | (1 << ISC00);

    EIMSK = (1 << INT1) | (1 << INT0);

    sei();
    atualizar_pwm();

    while (1) {}
}

```

4.

$$D_B = 100 - D_A$$

Exemplos:

D_A	D_B
60%	40%
30%	70%
80%	20%

```

#define F_CPU 4000000UL

#include <avr/io.h>

#define TOP_PWM 82

```

```

uint8_t duty_A = 60;
uint8_t duty_B = 40;

uint16_t calcular_ocr(uint8_t duty)
{
    if (duty > 100)
        duty = 100;

    return ((uint32_t)TOP_PWM * duty) / 100;
}

void atualizar_pwm(void)
{
    if (duty_A > 100)
        duty_A = 100;

    duty_B = 100 - duty_A;

    OCR1A = calcular_ocr(duty_A);
    OCR1B = calcular_ocr(duty_B);
}

int main(void)
{
    DDRB |= (1 << PB1) | (1 << PB2);

    ICR1 = TOP_PWM;

    TCCR1A = (1 << COM1A1) | (1 << COM1B1) | (1 << WGM11);
    TCCR1B = (1 << WGM13) | (1 << WGM12) | (1 << CS10);

    atualizar_pwm();

    while (1)
    {
        atualizar_pwm();
    }
}

```